Synthetic Financial Time Series Generation

GAN Benchmark: TimeGAN · QuantGAN · FinGAN

BRICS Emerging Market Indices

Collaboration: Universitat Politècnica de Catalunya (UPC)

Document type: Project Status & In-Depth Technical Review

Purpose: Paper publication preparation

Models covered: TimeGAN (RNN) · QuantGAN (TCN-WGAN-GP) · FinGAN (CNN-WGAN-GP)

Markets: BOVESPA · FTSE JSE · MSCI · NIFTY50 · SHANGHAI COMPOSITE

Research context · data pipeline · model status · open tasks

Research Goal

- Benchmark three GAN architectures for synthetic financial log-return generation
- Evaluate on 5 BRICS emerging-market indices (multi-market generalisation)
- Systematically validate hyperparameters via grid search for publication
- Compare stylized-fact reproduction: fat tails, volatility clustering, long memory
- Prepare peer-reviewed paper in collaboration with UPC

Data Pipeline

- Source: 5 CSV files — BOVESPA, FTSE JSE, MSCI, NIFTY50, SHANGHAI
- Feature: daily log return r_t = ln(P_t / P_{t-1})
- Outlier clipping at 0.5th / 99.5th percentile per market
- Temporal 80 / 10 / 10 split (no shuffle) — saved as Parquet files
- GAN training: windows of 128 steps pooled across all 5 markets

  (temporal order preserved within each window; windows shuffled across markets)

TimeGAN

- Yoon et al. (2019), NeurIPS
- 4-phase training
- GRU encoder / decoder
- BCE loss + moment matching
- MinMax [0,1] normalisation
- Current: epochs = 5 (too low)

QuantGAN

- Wiese et al. (2020), Quant. Finance
- WGAN-GP + TCN (causal dilated)
- Generator: FC → upsample → TCN
- Discriminator: TCN → pool → MLP
- Adam betas=(0, 0.9)
- n_critic=3, lambda_gp=10

FinGAN

- Custom CNN-WGAN-GP
- Generator: FC → 3× ConvTranspose1d
- Critic: Conv1d + LayerNorm
- seq_len must be divisible by 8
- Adam betas=(0, 0.9)
- n_critic=3, lambda_gp=10

Pipeline Status

- [DONE] Reproducibility seeds added to all GAN classes
- [DONE] Deprecated fillna() API fixed across all classes
- [DONE] Grid search (grid_search_gan) implemented and wired
- [DONE] FinancialMetrics rewritten with time-series-aware metrics
- [TODO] Re-run pipeline to regenerate results with updated metrics
- [TODO] Increase TimeGAN epochs (5 is far below the original paper)

Key References

- Yoon et al. (2019). Time-series GAN. NeurIPS 32.
- Wiese et al. (2020). Quant GANs. Quantitative Finance.
- Gulrajani et al. (2017). Improved WGAN. NeurIPS.
- Ang et al. (2023). TSGBench. PVLDB 17(3).
- Meldrum et al. (2025). Synthetic Data for Finance. arXiv:2510.26076.
- Takahashi & Mizuno (2025). Diffusion for Finance. Quant. Finance 25(10).

Understanding the data before understanding the models

What is a Financial Time Series?

A financial time series is simply a sequence of numbers measured over time. For a stock or index: P_1, P_2, P_3, ..., P_T where P_t is the closing price on day t.

Example: the BOVESPA index on five consecutive days might be:
[124,500 → 126,200 → 125,800 → 128,100 → 127,600]

Problem with prices directly: prices grow over time (have a trend), so comparing "day 1 volatility" to "day 1000 volatility" is misleading. Statistical models struggle with non-stationary data.

Why Log Returns? (Building the Feature)

Instead of prices, we compute the log return for each day:

r_t = ln(P_t / P_{t-1}) = ln(P_t) - ln(P_{t-1})

Why the logarithm?

1. Symmetry: a 10% gain then 10% loss ≠ zero in simple returns. Log returns are additive: r_{1→3} = r_{1→2} + r_{2→3} (exactly).

2. Approximate percentage change: for small moves, ln(P_t/P_{t-1}) ≈ (P_t - P_{t-1}) / P_{t-1} (simple return)

3. Stationarity: log returns fluctuate around zero with no long-run trend, making them much easier to model statistically.

From our example: r_2 = ln(126,200 / 124,500) ≈ +0.0135 (+1.35%)

The First Discovery: Returns Are NOT Normal

The simplest assumption: r_t ~ N(μ, σ²) independently.
This is the foundation of many classic financial models (Black-Scholes, etc.).

What Are Stylized Facts?

Cont (2001) coined the term "stylized facts": statistical properties that appear consistently across virtually all financial markets and time periods.

They are the fingerprint of financial returns. Any generative model that claims to produce realistic synthetic returns MUST reproduce them.

The five most important:

(1) Heavy tails: extreme events far more frequent than Gaussian predicts.

(2) Volatility clustering: "turbulent periods beget more turbulence". If |r_t| is large today, |r_{t+1}|, |r_{t+2}| tend to also be large. Visually: a time series of |returns| shows "bursts" of high activity.

(3) Long memory in |returns|: the correlation between |r_t| and |r_{t+k}| decays very slowly — still positive even at lag k = 100 days.

(4) Near-zero autocorrelation of r_t itself: you cannot predict tomorrow's direction from today's return (weak-form market efficiency).

(5) Negative skewness for equity indices: crashes are more extreme than rallies (asymmetric distribution).

Why Generate Synthetic Data?

If we already have real data, why generate synthetic data?

1. Data augmentation: deep learning models for finance need large datasets; real data is scarce (markets close each day).

2. Stress testing: banks need to test risk models under extreme scenarios that may not exist in historical data.

From the intuition (counterfeiter and detective) to the mathematics

The Core Idea — A Two-Player Game

Goodfellow et al. (2014) proposed a clever framework:
train TWO neural networks that compete against each other.

GENERATOR (G): takes random noise z as input → outputs a fake sample $\tilde{x}$.
Goal: produce fake samples so realistic that the Discriminator is fooled.

DISCRIMINATOR (D): takes a sample (real or fake) → outputs a number in [0,1].
Goal: correctly classify real samples (output → 1) from fake ones (output → 0).

They train simultaneously:

- D gets better at spotting fakes → G must produce better fakes.
- G gets better at fooling D → D must become more discerning.

This is the counterfeiter-and-detective analogy:
Counterfeiter (G) starts making crude fake banknotes.
Detective (D) easily spots them.
The counterfeiter studies the detective's feedback and improves.
Eventually, the counterfeiter's notes are indistinguishable from real.

For our problem: G generates fake log returns; D judges whether they look like real market returns.

The Mathematical Formulation

Let $p_{data}(x)$ = true distribution of real financial returns.
Let $p_g(x)$ = distribution of samples generated by G.

The GAN training is a minimax game (Goodfellow et al., 2014):

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{data}}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1 - D(G(z)))]$$

The Critical Problem: Training Instability

In practice, GAN training is notoriously difficult. Two main failure modes:

PROBLEM 1 — Mode Collapse:
G learns to output only a few samples that reliably fool D,
ignoring the diversity of the real distribution.
Analogy: the counterfeiter only produces 10-euro notes, not 20, 50, 100.

PROBLEM 2 — Vanishing gradients:
The GAN loss V uses Jensen-Shannon (JS) divergence internally.
When p_{data} and p_g have disjoint support (early in training),
JS divergence = log(2) (a constant) — no gradient information for G.

Analogy: if the counterfeiter's fakes are SO BAD that the detective immediately says "fake" without any useful feedback on HOW to improve, the counterfeiter makes no progress.

What does "disjoint support" mean?
If real returns cluster around [-0.05, +0.05] and G produces outputs in [-10, +10], there is no overlap → the JS distance gives no useful gradient telling G which direction to move.

What Makes GANs Suitable for Financial Returns?

Unlike parametric models (GARCH, EVT), a GAN:

- Makes NO distributional assumption about the shape of returns.
  It learns the distribution implicitly from data.

- Can model arbitrarily complex joint distributions $p(r_1, r_2, \ldots, r_T)$,
  capturing not just the marginal distribution but also temporal dependencies
  (volatility clustering, long memory).

Arjovsky et al. (2017) ICML · Gulrajani et al. (2017) NeurIPS

A Better Way to Measure "Distance" Between Distributions

The core insight of WGAN: the original GAN's failure comes from using
the WRONG measure of how different two distributions are.

The JS divergence (used implicitly in the original GAN) has a fatal flaw:
If p_data and p_g have disjoint support, JS(p_data, p_g) = log(2)
This is a CONSTANT — its derivative w.r.t. G's parameters is ZERO.
G receives no gradient signal and cannot learn.

Enter the Wasserstein-1 distance (also called "Earth Mover's distance"):

$$W(p, q) = \inf_{\gamma \in \Pi(p,q)} \; E_{(x,y)\sim\gamma} [\, |x - y| \,]$$

Think of it with a physical analogy:
Imagine p_data as a pile of sand shaped like the real distribution
and p_g as a pile of sand shaped like the fake distribution.
W is the minimum total "work" needed to reshape one pile into the other
— where work = (mass moved) × (distance moved).

Why is this better?
Even when distributions don't overlap at all,
W tells you HOW FAR APART they are and in which direction.
G always receives a meaningful gradient → no vanishing gradient problem.

The Kantorovich-Rubinstein Duality — Making W Computable

The inf over all joint distributions γ in W(p,q) is intractable directly.
The key mathematical result (Kantorovich-Rubinstein duality):

$$W(p_{data}, p_g) = \sup_{\|f\|_L \le 1} \; E_{x\sim p_{data}}[f(x)] - E_{x\sim p_g}[f(x)]$$

where the sup is taken over all 1-Lipschitz functions f.

Gradient Penalty: How to Enforce the Lipschitz Constraint

The original WGAN clipped weights to enforce Lipschitz (crude, slow).
WGAN-GP (Gulrajani et al., 2017) uses a smoother penalty instead:

$$GP = \lambda \cdot E_{\hat{x} \sim p_{\hat{x}}} [\, (\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2 \,]$$

where:
$\hat{x} = \varepsilon \cdot x_{real} + (1-\varepsilon) \cdot x_{fake}$ (a random interpolation)
$\varepsilon \sim U[0,1]$ (random mixing coefficient)
$\lambda = 10$ (penalty strength, standard value)

Reading the penalty:
$\|\nabla D(\hat{x})\|_2$ is the norm of D's gradient at the interpolated point.
For a 1-Lipschitz function, this norm should equal 1 everywhere.
The penalty pushes the gradient norm toward 1 throughout training.

Full training loss (for each step):
$L_D = E[D(\tilde{x})] - E[D(x)] + \lambda \cdot GP$ (D maximises -L_D)
$L_G = -E[D(G(z))]$ (G minimises L_G)

n_critic = 3: D is updated 3 times for each G update.
Ensures D stays near its optimum (approximates W well) before
G uses the gradient signal. This is critical for stability.

Adam Optimiser Settings for WGAN-GP

Recommended by Gulrajani et al. (2017): Adam(lr=1e-4, betas=(0, 0.9))

Standard Adam uses betas=(0.9, 0.999):
β1 = 0.9 → exponential moving average of GRADIENT (momentum).
β2 = 0.999 → exponential moving average of squared gradient (RMS scaling).

Why β1 = 0 for WGAN-GP?

Yoon, Jarrett & van der Schaar (2019) NeurIPS 32 — building on GRU + GAN

The Problem a Plain GAN Cannot Solve

A basic GAN generates each window independently from noise.
It can learn to match the MARGINAL distribution (heavy tails, right scale),
but it has no mechanism to enforce that consecutive timesteps within a window
exhibit temporal patterns like volatility clustering.

Analogy: asking someone to write a convincing novel by choosing
each word at random from a realistic vocabulary. The individual words
might all be real English, but the sequence makes no coherent sense.

What is a GRU? (From Scratch)

A GRU (Gated Recurrent Unit) is a neural network designed for sequences.
It processes one input at a time while maintaining a "hidden state" h_t,
like a reader maintaining a running summary of a book.

At each step t, the GRU receives:

- x_t — the current input (e.g. the return on day t)
- h_{t-1} — the hidden state summarising all previous inputs

And produces:

- h_t — updated hidden state

Internally, two "gates" control the information flow:

Reset gate r_t = σ(W_r · [h_{t-1}, x_t]) — how much of h_{t-1} to forget
Update gate z_t = σ(W_z · [h_{t-1}, x_t]) — how much of old h to keep
Candidate ñ_t = tanh(W · [r_t ∘ h_{t-1}, x_t])
New state h_t = (1-z_t) ∘ h_{t-1} + z_t ∘ ñ_t

Key property: h_t is a compressed summary of all inputs so far,
carrying information about temporal patterns across many timesteps.

TimeGAN's Key Innovation: Embedding Space

TimeGAN trains the GAN NOT in the raw return space X,

The Four-Phase Training Algorithm

Phase 1 — Autoencoder Training (epochs // 2 steps)
Embedder e(·): X → H (GRU: raw returns → latent representation)
Recovery r(·): H → $\tilde{X}$ (GRU: latent → reconstructed returns)
Loss: L_R = ||X - r(e(X))||²
Goal: learn a faithful compressed representation of the return sequences.

Phase 2 — Supervisor Training (epochs // 2 steps)
Supervisor s(·): H → $\hat{H}$ (GRU: predict next latent state)
Loss: L_S = ||H_{t+1} - s(H_t)||² (step-ahead prediction in latent space)
Goal: embed the temporal dynamics of real returns into the latent space.
Generator g(·) is co-trained here to have outputs in the same latent space.

Phase 3 — Joint Adversarial Training (epochs steps)
Generator: Z ~ N(0,I) → g(Z) = $\hat{E}$ → s($\hat{E}$) = $\hat{H}$ → r($\hat{H}$) = $\tilde{X}$
Discriminator: operates on the hidden states H (not raw returns)

Generator loss:
L_U = -E[log D(s(g(Z)))] (adversarial: fool discriminator)
L_S = ||H_{t+1} - s(H_t)||² (maintain temporal dynamics)
L_V = |μ_H - μ_{$\hat{H}$}| + |σ_H - σ_{$\hat{H}$}| (moment matching)
L_G = L_U + 100·sqrt(L_S) + 100·L_V

Discriminator loss:
L_D = BCE(D(H_real), 1) + BCE(D(H_fake), 0)

Critical Limitation: Training Budget

With the current setting of epochs = 5:
Phase 1 (autoencoder) runs for 5 // 2 = 2 gradient steps.
Phase 2 (supervisor) runs for 5 // 2 = 2 gradient steps.

2 gradient steps CANNOT train a GRU to convergence.

What is a Convolution? (From Scratch)

A 1D convolution slides a small "filter" (kernel) over a sequence, computing a weighted sum at each position:

$$out[t] = \sum_{k=0}^{K-1} w_k \cdot in[t - k]$$

where $w_0, ..., w_{K-1}$ are learnable weights (kernel size K).

Think of it like a magnifying glass that scans the time series, highlighting patterns of a specific shape and length.

Causal constraint — looking only into the past:

$out[t] = \sum_{k=0}^{K-1} w_k \cdot in[t - k]$

Position t can only see inputs at t, t-1, ..., t-(K-1).

No "future leakage" — important for financial time series modelling.

Dilated convolution — skip steps to see further back cheaply:

$out[t] = \sum_{k=0}^{K-1} w_k \cdot in[t - k \cdot d]$ (dilation d)

With d=1: looks at consecutive positions [t, t-1, t-2].

With d=2: looks at every other position [t, t-2, t-4].

With d=4: [t, t-4, t-8].

Stacking layers with dilation 1, 2, 4, 8, ... exponentially increases the receptive field (how far back a neuron can "see") without adding many parameters. For K=3, L=4 layers: receptive field = 1 + 2*(1+2+4+8) = 31.

QuantGAN — TCN-based WGAN-GP (Wiese et al. 2020)

Key innovation: replace GRU with TCN (parallelisable, no vanishing gradients).

Generator architecture:

z (noise_dim=100) → Linear → reshape (C, T/4)

→ ConvTranspose1d (stride=2, ×2 upsampling)

FinGAN — CNN Deconvolution WGAN-GP

A different convolutional approach: use transposed convolutions to directly "upsample" from a noise vector to a full time series.

Transposed convolution (ConvTranspose1d):

The "reverse" of a strided convolution.

Inserts zeros between inputs, then applies a convolution.

With stride=2: every input step produces 2 output steps.

Three such layers: T/8 → T/4 → T/2 → T (8× upsampling).

This is why seq_len must be divisible by 8.

Generator architecture:

z (noise_dim=100) → Linear+BN → reshape (C, T/8)

→ ConvTranspose1d(C/2, stride=2) → BN → LeakyReLU

→ ConvTranspose1d(C/4, stride=2) → BN → LeakyReLU

→ ConvTranspose1d(1, stride=2) → Tanh → $\tilde{x}$

Critic (discriminator) architecture:

x (T×1) → Conv1d(C) → LayerNorm → LeakyReLU

→ Conv1d(2C, stride=2) → LayerNorm → LeakyReLU

→ Conv1d(4C, stride=2) → LayerNorm → LeakyReLU

→ Flatten → Linear(128) → Linear(1)

LayerNorm (not BatchNorm) in the critic — as explained on page 5.

QuantGAN vs FinGAN vs TimeGAN: Comparison

Architectural philosophy:

TimeGAN: sequential (step-by-step); explicit temporal dynamics via GRU.
Most faithful to the sequential nature of time series.

QuantGAN: parallel (full window at once); long-range via dilation.
Faster training; larger receptive field without depth.

FinGAN: parallel; global upsampling from a single noise vector.

Cont (2001) Quantitative Finance · Mandelbrot (1963) · Ding, Granger & Engle (1993)

#	Stylized Fact	In Plain English	Mathematical Statement	Our Metric
1	Heavy tails	Extreme events occur far more often than a bell curve predicts.	P(|r|>x) ~ x^{-α}, α≈3–5 (Pareto) kurtosis(r) >> 3	Kurtosis diff · tail-index diff
2	Near-zero autocorrelation	Knowing today's direction gives no useful information about tomorrow's.	Corr(r_t, r_{t+k}) ≈ 0 for k≥1 (weak-form market efficiency)	ACF returns MAE
3	Volatility clustering	Large moves tend to be followed by large moves (Mandelbrot 1963).	Corr(|r_t|, |r_{t+k}|) > 0 for k = 1, ..., 100+	ACF |returns| MAE · ARCH-LM stat diff
4	Long memory in volatility	The clustering effect persists for hundreds of trading days.	ACF(|r_t|) ~ k^{-β}, β∈(0,1) Hurst exponent H > 0.5	Hurst exponent diff
5	Gain / loss asymmetry	Crashes are sharper and more extreme than equivalent-size rallies.	skewness(r) < 0 for equity indices; left tail heavier than right	Skewness diff
6	ARCH effects	Return variance is not constant — it changes over time.	Var(r_t | F_{t-1}) = σ²_t time-varying; not a constant σ²	ARCH-LM test (Engle 1982)
7	Conditional heavy tails	After removing time-varying variance, residuals are still non-Gaussian.	ε_t = r_t/σ_t still has kurtosis >> 3 (GARCH standardised residuals)	Kurtosis diff on GARCH residuals (ext.)
8	Extreme events	Very large days occur far more frequently than Gaussian probability predicts.	% of days with |r|>2σ exceeds 5% expected under N(0,1)	Extreme events diff
9	Distributional match	The full shape of the return distribution — not just its tails — must be reproduced.	W_1(p_g, p_data) ≈ 0 F̂_syn ≈ F̂_real (quantile-by-quantile)	Wasserstein · Quantile MSE · Energy distance
10	Indistinguishability	A classifier trained to separate real from synthetic should perform at chance level.	AUC(classifier) → 0.5 (p_g = p_data at GAN optimum)	Discriminative AUC

What Does i.i.d. Mean?

A sample {X_1, X_2, ..., X_n} is i.i.d. (independent and identically distributed) if:

- Identically distributed: every X_i has the same distribution F.
- Independent: knowing X_1, ..., X_{t-1} gives NO information about X_t.

Coin flips are i.i.d.: the 100th flip is unaffected by the previous 99.

Financial returns are NOT i.i.d.:

- The size of |r_{t+1}| is correlated with |r_t| (volatility clustering).
- Large shocks cluster together — ARCH effects mean Var(r_t|past) ≠ const.
- The autocorrelation of |r_t| is still positive at lag 100+ (long memory).

Most classical statistical tests are DERIVED under the i.i.d. assumption.
Applying them to time series requires understanding the consequences.

The Kolmogorov-Smirnov Test — What It Really Tests

The two-sample KS statistic:

$$D_{n,m} = \sup_x \left| \hat{F}_n(x) - \hat{G}_m(x) \right|$$

$\hat{F}_n$, $\hat{G}_m$ are empirical CDFs of two samples of size n and m.

Under H_0 (same distribution) AND i.i.d. observations:

$$\sqrt{nm/(n+m)} \cdot D_{n,m} \to K \quad \text{(Kolmogorov distribution)}$$

The p-value is derived from this limiting distribution.

For dependent series, the CLT underlying this limit changes:

$$\mathrm{Var}(n^{-1} \sum_t X_t) = (\sigma^2/n) \cdot (1 + 2 \cdot \sum_{k=1}^{\infty} \rho_k)$$

Welch's t-test — A Narrower Failure

Welch's t-test checks H_0: μ_real = μ_synthetic:

$$t = (\bar{X} - \bar{Y}) / \sqrt{s^2_X/n + s^2_Y/m}$$

For i.i.d. data: s^2_X/n consistently estimates $\mathrm{Var}(\bar{X}) = \sigma^2_X/n$.

For dependent data: the correct estimator is:

$$\mathrm{Var}(\bar{X}) = (\sigma^2_X/n) \cdot (1 + 2 \cdot \sum_{k=1}^{\infty} \rho_k(X))$$

For log returns, $\rho_k(r_t) \approx 0$ for $k \geq 1$,
so Welch's t-test is approximately valid for testing mean equality.

However, it tests ONLY the mean — providing no information about tails, volatility clustering, or any other stylized fact.

Using Welch's t-test as the primary evaluation metric (as was done in the original thesis for LLM evaluation) tells us almost nothing about whether the synthetic data is financially realistic.

What Can the KS Test Still Tell Us?

Despite its limitation, the KS test is not useless:

- It tests the MARGINAL distribution F(x) = P(r ≤ x).
- The marginal distribution is well-defined even for stationary dependent series (strict stationarity is sufficient).
- A GAN that produces the wrong scale, wrong sign, or obviously non-financial-looking returns will fail the KS test.

Strategy used in this project:

1. ARCH-LM Test — Does the GAN Reproduce Volatility Clustering?

Problem it solves: the KS test cannot detect whether synthetic returns have volatility clustering. A GAN could pass KS (correct marginal) but generate i.i.d. returns — no clustering.

What it tests: H_0 = no ARCH effects (homoskedasticity).

Procedure (Engle 1982):

(1) Regress squared returns on their own lags:

$\hat{\varepsilon}^2_t = \alpha_0 + \alpha_1 \cdot \hat{\varepsilon}^2_{t-1} + ... + \alpha_q \cdot \hat{\varepsilon}^2_{t-q} + v_t$

(2) Compute LM = n · R^2 where R^2 is from this regression.

(3) Under H_0: LM ~ $\chi^2(q)$

(4) If p < 0.05: reject H_0 → ARCH effects present.

How we use it:

Real BRICS returns: consistently show ARCH (p << 0.05).

Synthetic from a good GAN: should also show ARCH.

Our metric: |LM_real - LM_synthetic| → 0 is ideal.

We also check arch_reproduced: does the YES/NO match?

2. Hurst Exponent — Does the GAN Reproduce Long Memory?

Problem it solves: detects whether |returns| have the slow ACF decay (long memory, stylized fact 4) that real financial series exhibit.

Physical intuition: the Hurst exponent H characterises the scaling behaviour of the "rescaled range" R/S of a time series.

E[R_n / S_n] ~ c · n^H

where R_n = range of partial sums, S_n = standard deviation.

Interpretation:

H = 0.5 → Brownian motion (independent increments, no memory).

3. Energy Distance — A Proper Distributional Metric

The energy distance between distributions P and Q (Székely & Rizzo 2004):

$E(P,Q) = 2 \cdot E||X-Y|| - E||X-X'|| - E||Y-Y'||$

where X,X'~P and Y,Y'~Q are independent copies.

Plain English: it measures the average distance between a random sample from P and one from Q, corrected for within-distribution spread.

Key properties:

- E(P,Q) ≥ 0, with equality iff P = Q. A true metric on distributions.
- Works on any dimension; does not require a density function.
- Does not assume independence: uses marginal expectations.

Estimated using n=min(500,N) subsampled pairs ($O(n^2)$ cost, fast in practice).

Advantage over Wasserstein: simpler to estimate in practice; advantage over KS: a proper metric, not just a supremum distance.

4. Discriminative AUC — The Holistic Score

The gold-standard evaluation for GANs: can a classifier tell real from fake?

Procedure (TSGBench, Ang et al. 2023):

1. Extract rolling-window features from both real and synthetic: (mean, std, skewness, kurtosis, |r| mean, |r| max) per window.
2. Label real windows 0, synthetic windows 1.
3. Train logistic regression classifier.
4. Report AUC from 5-fold cross-validation.

AUC = 0.5 → classifier performs at chance → distributions indistinguishable.

AUC = 1.0 → trivially separable → GAN has failed.

3_4_integrated_pipeline.ipynb — reviewed and updated July 2026

FIXED	**KS / Welch tests assumed i.i.d. — CRITICAL** n_eff = n/(1+2Σρ_k) << n for financial series (sum of ACF can be 5–15 for |returns|). This inflates the KS statistic, making p-values anti-conservative. Fix: added ARCH-LM test, Hurst exponent, energy distance, and discriminative AUC. KS retained with caveat; Wasserstein used as the primary distributional metric.
FIXED	**Pointwise MSE/MAE had no semantic meaning** MSE compared r_real(t) to r_syn(t) but real and synthetic are independently generated: no temporal alignment exists between their indices. Fix: replaced with Quantile MSE (L2 distance between empirical quantile functions) and Energy distance (Székely & Rizzo 2004 — a proper metric on the space of distributions).
FIXED	**Ljung-Box test used deprecated statsmodels API** acorr_ljungbox(..., return_df=False) returns a DataFrame in statsmodels >= 0.13, not a tuple. Accessing lb[1][-1] raised TypeError. Fix: return_df=True, lags=[n] (list syntax), accessed via .iloc[-1]["lb_pvalue"].
FIXED	**ResidualBlock class defined but never used in FinGAN cell** class ResidualBlock(nn.Module) was defined at the top of the FinGAN cell but neither FinGAN_Generator nor FinGAN_Critic references it anywhere. Dead code creates confusion for readers who may think residual connections are active. Fix: class removed entirely.
FIXED	**Grid search score_cols included pointwise MSE and MAE** The composite ranking used to select hyperparameter configurations included "mse" and "mae" — which as described above compare unaligned series (meaningless). Fix: removed mse/mae from score_cols; added hurst_diff, energy_distance, arch_stat_diff.
OK	**WGAN-GP gradient penalty computation is correct (QuantGAN & FinGAN)** x̂ = ε·x_real + (1-ε)·x_fake; x̂.requires_grad_(True); gradients computed w.r.t. x̂ via torch.autograd.grad with create_graph=True. fake.detach() ensures fake does not accumulate spurious gradients. This correctly implements Gulrajani et al. (2017) Eq. 3.
OK	**Adam betas=(0.0, 0.9) for WGAN-GP — correct** β1=0 disables first-moment (momentum) accumulation. In adversarial training, gradients change direction frequently; momentum from the previous step destabilises training. β2=0.9 retains RMS adaptive scaling. Recommended explicitly by Gulrajani et al. (2017).
CONCERN	**TimeGAN training budget of epochs=5 is scientifically insufficient** Phase 1 (autoencoder) gets 5//2=2 gradient steps; Phase 2 (supervisor) gets 2 steps. A GRU cannot converge in 2 steps. The embedding space H is meaningless → Phase 3 trains on noise. Yoon et al. (2019) use 5,000–10,000 iterations per phase. TimeGAN's poor ranking almost certainly reflects training budget, not architecture. Requires fix.

Required for publication: systematic evidence that parameters were tested before selection

Why Grid Search is Required for Publication

A common criticism in ML papers: "how were hyperparameters chosen?"
If the answer is "we tried a few and picked the best-looking result",
reviewers rightly question whether the result is reproducible or cherry-picked.

Grid search provides a paper trail:

- The search space is defined ex-ante (before seeing results).
- Selection criterion is objective (composite rank of stylized-fact metrics).
- Every candidate configuration is documented in a CSV (appendix).
- The winning configuration is then retrained at full epochs.

This turns hyperparameter choice from a hidden decision into a
documented, reproducible scientific step.

The Search Protocol

For each (model, config) combination:

1. Set config[epochs] = 3 (reduced budget for speed)
   Set config[seed]   = 42 (same seed for all candidates)
2. Train model on pooled BRICS training windows.
3. Generate synthetic series of same length as validation set.
4. Score with FinancialMetrics.compute_all_metrics().
5. Rank across all candidates on each metric (rank 1 = best).
6. composite_score = mean(all ranks).
7. Best config → retrain at full epochs (5) for final results.

Why search_epochs=3 < final_epochs=5?
Reduced budget (coarse-to-fine) is standard practice (Bergstra 2012).
At 3 epochs each model produces a rough relative ranking;
the cardinal value of the score does not need to be precise.

Composite Scoring Formula

For each metric m ∈ score_cols and each candidate config_i:

rank_m(config_i) = rank by |metric_m(config_i)| (1 = best, 8 = worst)

Search Space (8 candidates = 2^3 per model)

TimeGAN: hidden_dim ∈ {24, 48} · num_layers ∈ {2, 3} · lr ∈ {1e-3, 5e-4}
QuantGAN: lr ∈ {1e-4, 2e-4} · n_critic ∈ {3, 5} · lambda_gp ∈ {5, 10}
FinGAN: lr ∈ {1e-4, 2e-4} · n_critic ∈ {3, 5} · base_channels ∈ {32, 64}

Fixed across all: noise_dim=100 (QuantGAN/FinGAN), noise_dim=24 (TimeGAN).
Batch size: 128 (TimeGAN), 64 (QuantGAN/FinGAN).
All candidates use seed=42 — ensures identical initialisation for fair comparison.

Reproducibility

Each candidate calls torch.manual_seed(seed) + np.random.seed(seed).
Same config + same seed → identical weights and training trajectory.
Results saved to: grid_search_results/{Model}_grid_search.csv
  Columns: all searched params + all metric values + composite_score.
  This CSV is the appendix for hyperparameter documentation in the paper.

Recommended Extensions for the Paper

- Extend to 20–30 configs × 10 epochs per candidate (currently 8 × 3).
- Use Bayesian optimisation (Optuna) for more efficient exploration.
- Ablation table: vary one parameter at a time, fix all others;
  report the marginal effect on composite score.
- 5-fold cross-validation across market splits for robust scoring.
- Report the best config with 95% CI on composite score across seeds.

Priority order based on impact on scientific rigour and reviewability

CRITICAL	**1 — Re-run the full pipeline and regenerate all results** Evaluation metrics schema changed (ARCH-LM, Hurst, discriminative AUC, energy distance added; pointwise MSE/MAE removed from ranking). All files in thesis_results/ are stale. Must regenerate before any comparative analysis or figure creation for the paper.
CRITICAL	**2 — Fix TimeGAN training budget (epochs=5 is scientifically unsound)** Phases 1 and 2 each get 2 gradient steps. The Yoon et al. (2019) paper uses 5,000–10,000 iterations per phase. TimeGAN's poor ranking almost certainly reflects this, not its architecture. Recommendation: set epochs ≥ 50 or add explicit per-phase iteration counts.
HIGH	**3 — Robustness test on a different asset class (FX or Crypto)** Supervisor request: train the best GAN (QuantGAN) on BRICS, then evaluate on EUR/USD or BTC/USD daily log returns. This validates out-of-distribution generalisation — a key contribution that differentiates the paper from single-asset GAN studies.
HIGH	**4 — Add or discuss diffusion model baseline** Takahashi & Mizuno (2025, Quant. Finance 25(10)) showed diffusion models outperform GANs on several stylized-fact metrics. Reviewers will ask why diffusion was not included. At minimum: dedicated Discussion section. Best: include a simple DDPM or score-matching baseline using an open-source implementation (e.g. denoising-diffusion-pytorch).
HIGH	**5 — Document all LLM parameters for reproducibility** DeepSeek experiments: report temperature, top_p, max_tokens, system prompt, user prompt. LoRA fine-tuning: r=8, alpha=16, target=[q_proj,v_proj], dropout=0.1, training epochs, batch size, gradient accumulation. Upload code to Papers with Code (paperswithcode.com).
MEDIUM	**6 — Expand grid search and publish full ablation table** Current: 8 configs × 3 epochs. For publication: extend to 20–30 configs × 10 epochs; present ablation table showing marginal effect of each hyperparameter. This turns the grid search from a box-checking exercise into a publishable analysis.
MEDIUM	**7 — Update literature review with 2024/2025 papers** Must cite: Meldrum et al. (2025) arXiv:2510.26076; SFAG (2026) arXiv:2601.12990; TSGBench Ang et al. (2023) PVLDB 17(3); Chronos Ansari et al. (2024) arXiv:2403.07815; LLMTime Gruver et al. NeurIPS 2023; Forging TS Hamdouche et al. (2025) arXiv:2505.17103.
LOW	**8 — LLM diversity analysis: correlation between independent generation runs** Generate 10+ independent synthetic series from each LLM model. Compute pairwise Pearson correlation and Wasserstein distances. Mode collapse → high correlation. Diversity → low. This addresses the supervisor's question about LLM output diversity.
LOW	**9 — Fix cosmetic filename strip in validate_models()** validate_models() calls replace("_val.parquet","") but files are named "_valid.parquet". Display names appear as "BOVESPA_valid" instead of "BOVESPA". One-line fix; no effect on results.